

A Comprehensive Measure of Well-Being: HDI Prediction using AI/ML

Project Description:

A Comprehensive Measure of Well-Being harnesses Machine Learning to predict the Human Development Index (HDI) of a country based on key socioeconomic indicators, offering users a fast and data-driven way to assess human development levels. The platform includes an HDI Prediction Engine, which estimates the HDI score using Life Expectancy, Mean Years of Schooling, Gross National Income (GNI) per capita, and Internet Users as input, an HDI Category Classifier, which labels the result as Low, Medium, High, or Very High HDI, and an interactive Dashboard suite offering visual insights into global development trends.

Utilizing a Linear Regression model trained with Scikit-learn on the Kaggle HDI dataset, the application processes user inputs to deliver instant, interpretable predictions. Built with Flask and a clean web interface, the platform ensures a seamless and user-friendly experience. With organized data preprocessing, model persistence via Pickle, and three dedicated dashboards (HDI Comparison, World HDI Map, and HDI Trend Over Years), the application empowers researchers, students, and policy analysts to explore and understand human development patterns with confidence.

Scenarios

Scenario 1:

A researcher enters Life Expectancy of 72.5 years, Mean Years of Schooling of 10.2, GNI per capita of 15,000, and Internet Users of 65.4%. The system instantly predicts an HDI score of 0.76 and classifies it as High HDI, helping the researcher quickly assess a hypothetical country's development level.

Scenario 2:

A student studying global development opens the HDI Dashboard to compare the Top 20 and Bottom 10 countries by HDI score, instantly visualizing global disparities through interactive bar charts without manually processing the dataset.

Scenario 3:

A policy analyst uses the World HDI Map dashboard to filter countries by HDI category (Very High, High, Medium, Low) and reviews the HDI Trend Over Years chart to understand how regional development has evolved from 1990 to 2015, supporting evidence-based policy discussions.

Technical Architecture:

Description:

The HDI Prediction System utilizes a modular three-tier architecture to deliver fast, ML-driven predictions and visual insights. The frontend provides a responsive user interface built with HTML and CSS, while the Flask-based backend handles input validation, model inference, and routing. The trained Linear Regression model, serialized using Pickle, is loaded at application startup and used to generate predictions, with deployment managed locally via the Flask development server.

```
HDI Prediction System
|-- Built With --> Flask: Web Application Framework
|-- Receives --> User Input (Life Expectancy, Schooling, GNI, Internet Users)
|-- Integrate with --> Trained ML Model (HDI.pkl - Linear Regression)
    |-- Generates --> HDI Score Prediction
    |-- Generates --> HDI Category Classification
    |-- Generates --> Dashboard Visualizations (Comparison, Map, Trend)
|-- Ensure --> Model Persistence via Pickle (HDI.pkl)
```

(Note: This project uses a CSV dataset and a serialized Pickle model file instead of a traditional database; an Entity-Relationship Diagram describing the data structure is included separately in the Project Design Phase documentation.)

Pre-requisites:

- Flask Framework Knowledge: [Flask Documentation](#)
- Scikit-learn & Machine Learning Familiarity: [Scikit-learn Documentation](#)
- HTML, CSS, and JavaScript Skills: [W3Schools HTML/CSS/JavaScript Tutorials](#)
- Python Programming Proficiency: [Python Documentation](#)
- Version Control with Git: [Git Documentation](#)
- Development Environment Setup: [Anaconda / VS Code / Jupyter Notebook](#)
- Data Analysis Libraries: [NumPy, Pandas, Matplotlib, Seaborn Documentation](#)

Project Workflow:

Milestone 1: Data Collection & Environment Setup

- Activity 1.1: Set up Python environment using Anaconda and install required libraries (NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, Flask).
- Activity 1.2: Create the project folder structure with separate directories for Dataset, Training, and Flask modules.
- Activity 1.3: Download the Human Development Index dataset from Kaggle and load it into the development environment.
- Activity 1.4: Explore and understand the dataset structure, features, target variable (HDI), and data types using `head()`, `shape`, and `describe()` functions.

Milestone 2: Data Preprocessing & Model Training

- Activity 2.1: Select dependent (HDI score) and independent variables (Life Expectancy, Mean Years of Schooling, GNI per capita, Internet Users) for model training.
- Activity 2.2: Check for and handle missing values by filling null entries with column mean values.
- Activity 2.3: Perform data visualization including strip plots and a correlation heatmap to understand relationships between indicators and HDI.
- Activity 2.4: Split the processed dataset into training (90%) and testing (10%) sets using `train_test_split`.
- Activity 2.5: Train a Linear Regression model using the prepared training dataset and evaluate performance using R-squared score.
- Activity 2.6: Save the trained model using Pickle serialization (`HDI.pkl`) for reuse without retraining.

Milestone 3: app.py Development

- Activity 3.1: Write the main Flask application logic in `app.py`, establishing routes for the home page, prediction form, and prediction handler.

Define the Core Routes in `app.py`:

- `/` — Home page route that renders `home.html`
- `/Prediction` — Renders the input form (`indexnew.html`)
- `/predict` — POST endpoint that reads form values, converts them to numerical format, passes them to the trained model, and returns the classified HDI result
- `/Dashboard` — Renders the `dashboard.html` page containing the HDI Comparison, World Map, and Trend visualizations

```
@app.route('/')# route to display the home page
def home():
    return render_template('home.html') #rendering the home page
```

```

@app.route('/Prediction',methods=['POST','GET'])
def prediction():
    return render_template('indexnew.html')

@app.route('/Home',methods=['POST','GET'])
def my_home():
    return render_template('home.html')

@app.route('/predict',methods=['POST'])# route to show the predictions in a web UI
def predict():
    #reading the inputs given by the user
    input_features = [float(x) for x in request.form.values()]
    features_value = [np.array(input_features)]

    features_name = ['Life expectancy','Mean years of schooling','Gross national income
(GNI) per capita','Internet users']

    df = pd.DataFrame(features_value, columns=features_name)

    # predictions using the loaded model file
    output = model.predict(df)
    print(round(output[0],2))
    print(type(output))
    y_pred = round(output[0],2)
    if(y_pred >= 0.3 and y_pred <= 0.4):
        return render_template("resultnew.html",prediction_text = 'Low HDI'+ str(y_pred))
    elif(y_pred >= 0.4 and y_pred <= 0.7):
        return render_template("resultnew.html",prediction_text = 'Medium HDI
'+str(y_pred))
    elif(y_pred >= 0.7 and y_pred <= 0.8):
        return render_template("resultnew.html",prediction_text = 'High HDI'+str(y_pred))
    elif(y_pred >= 0.8 and y_pred <= 0.94):
        return render_template("resultnew.html",prediction_text = 'Very High
HDI'+str(y_pred))
    else:
        return render_template("resultnew.html",prediction_text = 'The given values do not
match the range of values of HDI')
@app.route('/Dashboard')
def dashboard():
    return render_template('dashboard.html')

if __name__ == '__main__':
    # running the app
    app.run(debug=True,port=5000)

```

Apply input validation: the form requires all four numerical fields before submission, and out-of-range predictions return a clear message indicating the values do not match expected HDI ranges.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clean, professional, and responsive interface for the HDI Prediction System, including the home page, prediction form, result page, and the dashboard suite with interactive Chart.js visualizations.

Set Up the Base HTML Structure:

- Developed home.html as the landing page introducing the HDI Prediction System with a call-to-action to start a prediction.
- Developed indexnew.html containing the input form for Life Expectancy, Mean Years of Schooling, GNI per capita, and Internet Users.
- Developed resultnew.html to display the predicted HDI score and category in a clean result card.
- Developed dashboard.html containing three sections: HDI Score Comparison, World HDI Map, and HDI Trend Over Years.

Design a Responsive Layout Using CSS:

- Implemented a clean, professional theme using CSS with consistent color palette and typography across all pages.
- Used flexbox and grid layouts to organize dashboard charts into a clear multi-section structure.
- Ensured responsive design across desktop and mobile screen sizes.

Create Dashboard Visualizations:

- HDI Score Comparison: Bar charts displaying Top 20 and Bottom 10 countries by HDI score, built using Chart.js.
- World HDI Map: Filterable country cards by HDI category, supported by Pie and Doughnut charts summarizing category distribution.
- HDI Trend Over Years: Line chart visualizing HDI trends by region from 1990–2015, with a radar chart for multi-indicator comparison.

Note: The dashboard concept and overall planning were led by Spandhana Mondam (Team Lead). The coding and implementation of the dashboard visualizations were carried out collaboratively and equally by the remaining team members.

Milestone 5: Local Deployment & Testing

Milestone 5 focuses on deploying the application locally to verify correct operation, and conducting functional and performance testing to ensure reliability before final submission.

Run the Flask Development Server:

```
cd ML-0027-Human-Development-Index/Flask
python app.py

* Running on http://127.0.0.1:5000
```

Local Testing and Verification:

- Opened the browser and navigated to `http://127.0.0.1:5000` to access the application.
- Tested the prediction form with multiple valid input combinations to verify correct HDI score and category classification.
- Verified the Dashboard page loads all three visualizations (Comparison, World Map, Trend) correctly with no rendering errors.
- Conducted performance testing to confirm response times remain under 2 seconds for predictions, as documented in the Performance Testing report.

Exploring the Website's Web Pages:

Home Page:

Description:

The landing page introduces the Human Development Index (HDI) Prediction System with a clean, professional design. It displays the project title, a short description of the application's purpose, and a 'Start Prediction' button that navigates the user to the input form.

Input / Prediction Page:

Description:

A clean form-based interface where users enter four numerical indicators: Life Expectancy, Mean Years of Schooling, Gross National Income (GNI) per capita, and Internet Users. Each field includes a placeholder example and is required before submission. Clicking 'Predict HDI' sends the data to the backend for processing.

Result Page:

Description:

Displays the predicted HDI score along with its classified category (Low, Medium, High, or Very High HDI) in a highlighted result card. The page includes options to predict again or return to the home page.

Dashboard Page:

Description:

An interactive analytics page divided into three sections. The HDI Score Comparison section shows Top 20 and Bottom 10 countries by HDI using bar charts. The World HDI Map section presents filterable country cards by HDI category alongside Pie and Doughnut charts. The HDI Trend Over Years section displays a line chart of HDI progression from 1990–2015 by region, plus a radar chart comparing multiple indicators across regions.

Conclusion:

A Comprehensive Measure of Well-Being is a Machine Learning-powered platform built with Flask that streamlines the prediction and understanding of Human Development Index scores. It uses a Linear Regression model trained on the Kaggle HDI dataset to instantly predict and classify development levels based on Life Expectancy, Education, Income, and Internet access indicators. The project, which included data analysis, model training, Flask backend development, and an interactive dashboard suite, highlights how data-driven tools can make complex socioeconomic indicators accessible and easy to interpret for researchers, students, and policy analysts. Looking ahead, the platform offers opportunities for expansion, such as incorporating additional development indicators, supporting historical year-wise predictions, and enabling country-specific comparisons. By continuing to refine the underlying model and enhancing the dashboard's interactivity, the platform is well-positioned to evolve into a comprehensive human development analytics tool.